TP 2 - Image Formation

Goal: to study image formation by projecting a 3D point cloud to an image plane using the pinhole camera model.

Main steps of the practical work:

1. Implement the pinhole projection to project every point in the object to the camera plane (Eq. 3)

After reading the pointcloud and assuming that the object is placed in a nice position, we need to obtain camera plane coordinates. We can form an inverted image by following the equations and considering that we don't need rigid transformation yet, so $Xp' = Xp;\ Yp' = Yp$:

$$Xcam = \frac{Xp'}{1 + \frac{Zp'}{f}};\ Ycam = \frac{Yp'}{1 + \frac{Zp'}{f}} \quad (1)$$

```
float X_p = points[i].x;
float Y_p = points[i].y;
float Z_p = points[i].z;


float X_cam, Y_cam;
if (orthogonal) {

} else {
    float denominator = 1.0 + Z_p/f;
    if (denominator == 0.0) continue;
    X_cam = X_p / denominator;
    Y_cam = Y_p / denominator;
}
```

2. Implement the uv projection (Eq. 4) to obtain pixel coordinates.

Next, we need to choose some camera intrinsic parameters such as image origin $u0,\ v0$ - which are basically coordinates of the image center and image resolution αu and αv (transforming meters to pixels). We can calculate pixel coordinates by formulas:

$$u = \frac{Xcam}{\alpha u} + u0;\ v = \frac{Ycam}{\alpha v} + v0 \quad (2)$$

```
int u = (int) roundf(X_cam / alpha_u + u_0);
int v = (int) roundf(Y_cam / alpha_v + v_0);
```

3. Fill the image coordinates with the point color. This should create an image.

We start with allocating memory needed for the image, throwing an error if we couldn't do it and then setting all pixels to be white (it will be easier to interpret cube transformations on the white background).

```
image.pixmap = (pixel *) malloc(image.cols * image.rows *
sizeof(pixel));
    if (!image.pixmap) {
        fprintf(stderr, "Couldn't allocate memory for the image");
        exit(1);
    }

    for (int i = 0; i < image.rows * image.cols; i++) {
        image.pixmap[i].red = image.pixmap[i].green =
image.pixmap[i].blue = image.maxval;
    }
```

Considering frameCube.off format, we can extract RGB color values, we can fill in the pixel coordinates we obtained previously. We should also check the bounds - those cordinates should be positive and within image borders. We don't use depth buffer yet, so we just overwrite the pixel in case if several points map to the same pixel.

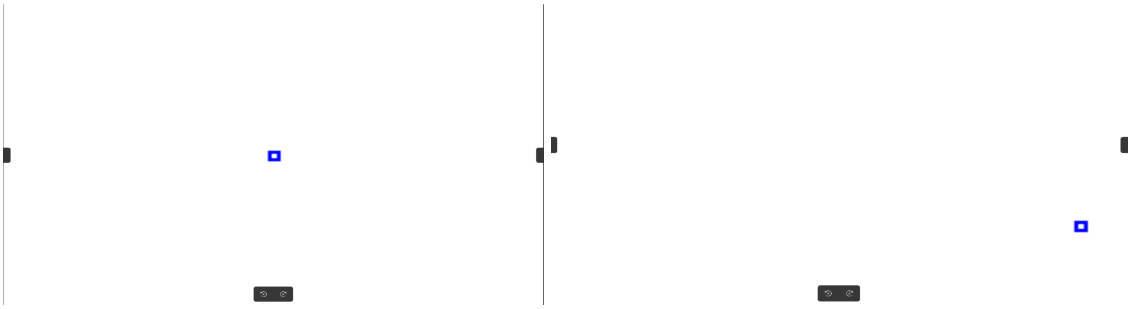
```
if (u >= 0 && u < width && v >= 0 && v < height){
                image.pixmap[u + v*width].red = points[i].r;
                image.pixmap[u + v*width].green = points[i].g;
                image.pixmap[u + v*width].blue = points[i].b;

        }
```

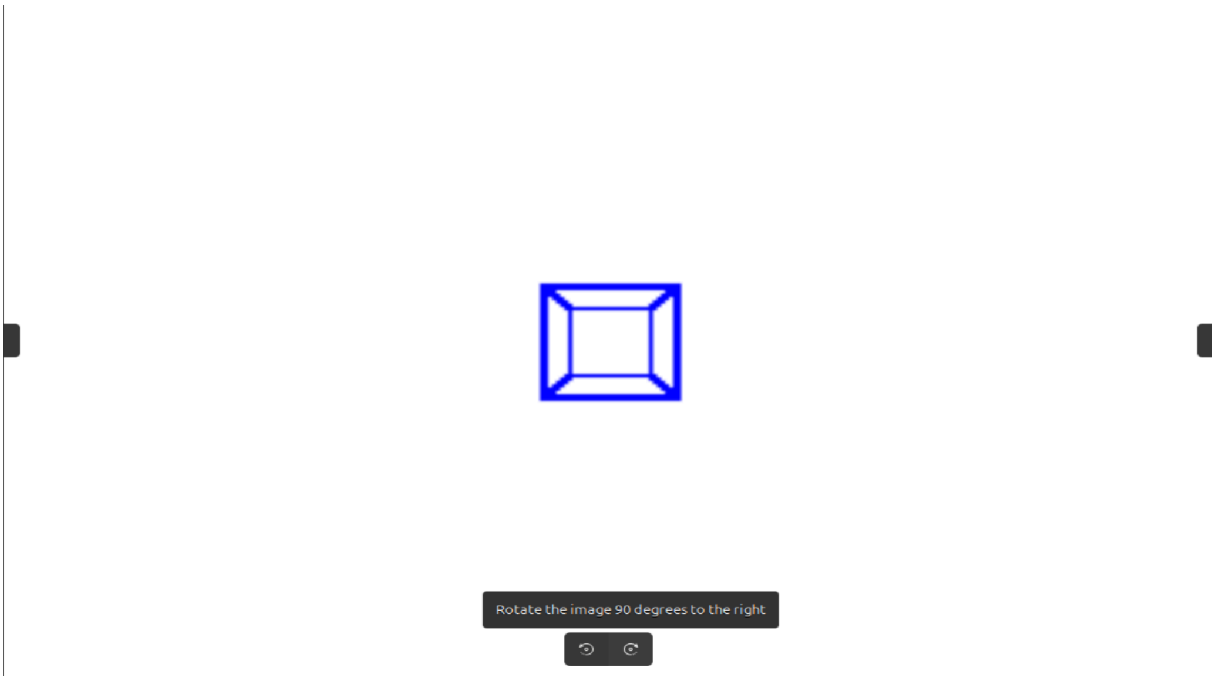
4. How does the image change if we modify the intrinsic parameters?
Now we want to move the camera, which is done with the repositioning of the cube.

We'll change the parameters one by one, to see the difference. Below, on the left there's a basic configuration (calling function % ./project_points models/cubeFrame.off 1 300 200 150 100 0.1 0.1 0 0 0 0 0 -2 0), so its image origin $u0 = 150$, $v0 = 100$, those are the halves of the width and height respectively, therefore, the image is located in the center. On the right,

$u0 = 275$, $v0 = 150$ so the cube is situated in the bottom right corner. So $u0$ is responsible for horizontal location, $v0$ - for vertical.



If we change image resolution αu and αv, we can scale image. In basic function call, scaling factors are $\alpha u = 0.1$ and $\alpha v = 0.1$, but the image is quite far and cube is small. To zoom in, we can divide these parameters by 5 to get $\alpha u = 0.02$ and $\alpha v = 0.02$, so the image becomes 5 times bigger. From formula (1) we remember that these parameters are in denominator so the smaller they are, the bigger zoom-in we get and vice versa.



And if we use primary $\alpha u = 0.1$ but decrease $\alpha v = 0.01$, the cube will flatten horizontally (the zoom in across vertical axis was 10 times bigger than horizontal axis).



5. Implement the rigid transformation of the 3D model (Eq. 1) and apply it to the points after centerThePCL and before the pinhole projection.

Now we're going to move and rotate the camera and use extrinsic parameters using the formula below:

$$\begin{pmatrix} X_{p'} \\ Y_{p'} \\ Z_{p'} \\ 1 \end{pmatrix} = \begin{pmatrix} \mathbf{R} & t \\ \mathbf{0} & 1 \end{pmatrix} \begin{pmatrix} X_p \\ Y_p \\ Z_p \\ 1 \end{pmatrix}$$

We're going to define a transformation matrix $T = \begin{pmatrix} \mathbf{R} & t \\ \mathbf{0} & 1 \end{pmatrix}$ (in the code it's an array), where R is a 3x3 rotational matrix and t a 3x1 translation vector. This matrix is built by computeTrans() function, where we pass extrinsic parameters as arguments. So we can transform a point into camera space and then implement pinhole projection with points in camera coordinate system.

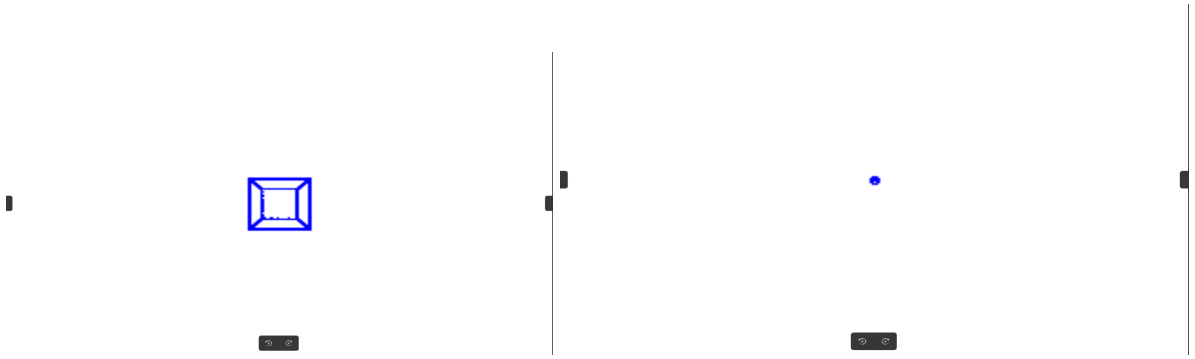
```
float X_p_cam = T[0]*X_p + T[1]*Y_p + T[2]*Z_p + T[3];
float Y_p_cam = T[4]*X_p + T[5]*Y_p + T[6]*Z_p + T[7];
float Z_p_cam = T[8]*X_p + T[9]*Y_p + T[10]*Z_p + T[11];

float X_cam, Y_cam;
if (orthogonal) {
```

```
        } else {
            float denominator = 1.0 + Z_p_cam/f;
            if (denominator == 0.0) continue;
            X_cam = X_p_cam / denominator;
            Y_cam = Y_p_cam / denominator;
```

6. How does the image change if we modify the extrinsic parameters?
Now we want to implement the orthogonal projection. Use f = 0 in the code.

The first parameter is γ ($gama$) - responsible for rotation around Z axis. Below, on the left $\gamma = -1.57\ (-90\ degrees)$, cube spins here like a flat square. On the right, $\gamma = 0.52\ (30\ degrees)$ - strangely enough, cube looks like a circle because of small focal length f and distortion.



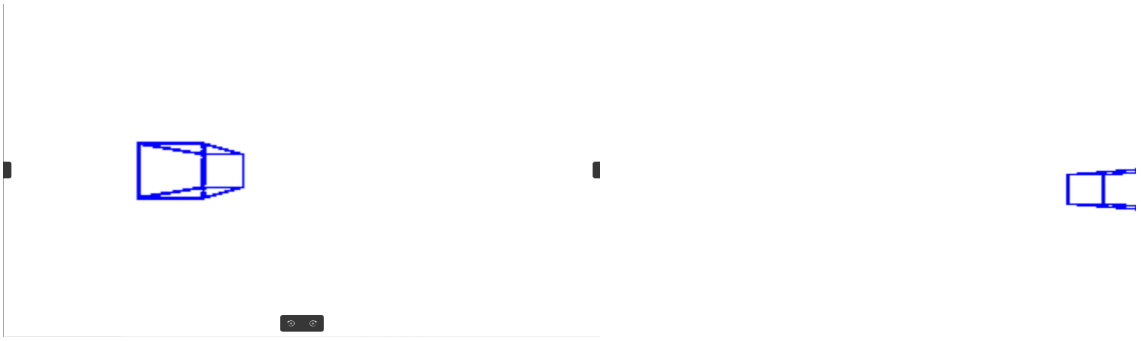
The second parameter is β ($beta$) - responsible for rotation around Y axis. Cube is rotating sideways. Below, on the left $\beta = 0.79\ (45\ degrees)$, the image is turning right. On the right, $\gamma = -0.52\ (-30\ degrees)$, the image is turning left.



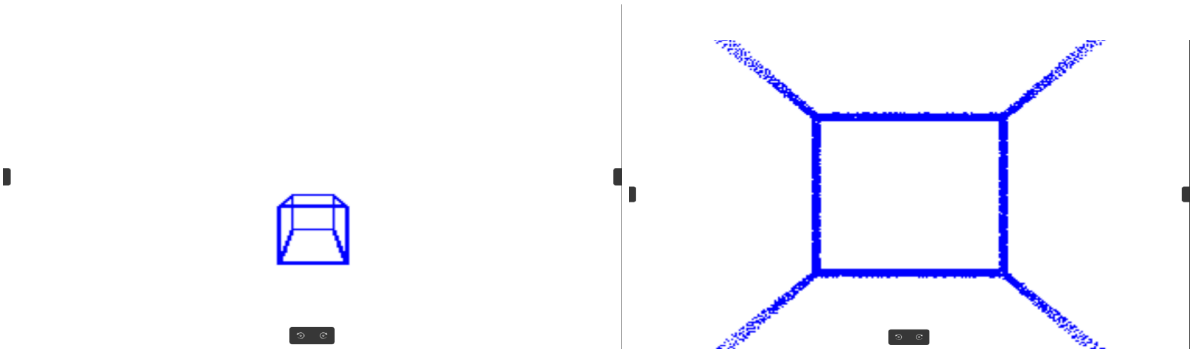
The third parameter is α ($alpha$) - responsible for rotation around X axis. Cube is rotating up and down. Below, on the left $\alpha = 0.52\ (30\ degrees)$, we can see the top face of the cube. On the right, $\alpha = 1.05\ (60\ degrees)$, it's even more visible.



Parameters Tx, Ty are moving the object sideways/up and down respectively. On the left, $Tx = 1$, so the object is translated to the left; on the right, $Tx = -3$, and the object is translated to the right and it's almost out of virewpoint of the camera.



Similarily, on the left, $Ty = -0.5$ - the object is translated to the down. Parameter Tz translates the cube along Z axis, more precisely, it controls how far the object is from the camera. We always used $Tz = -2$ which moved the object in front of the camera. But if we diminished it to $Tz = -1$, we can observe how closer it became to the camera (on the right below).

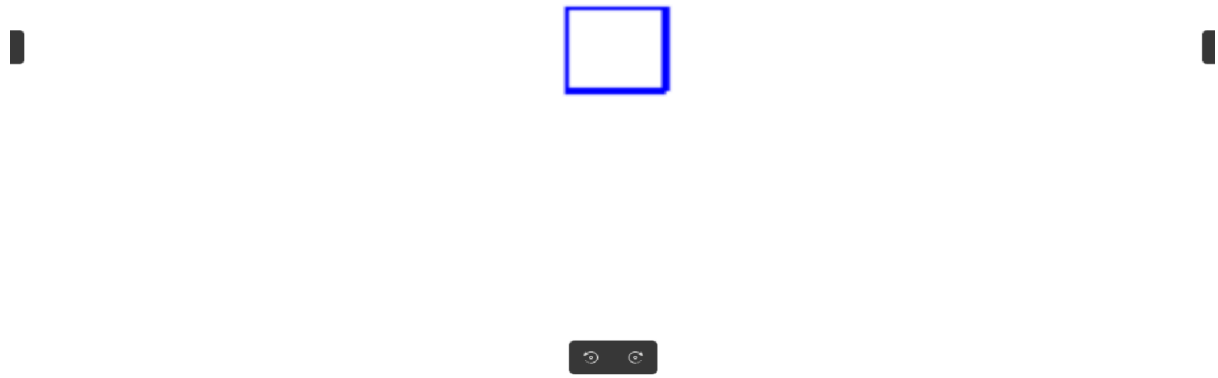


Furthermore, if Tz becomes too negative, it'll be farther away but if it reaches 0 or becomes positive, it'll be invisible or distorted as the cube will be at the camera center or even behind the camera.

7. Implement orthogonal projection (you probably need to change the resolution a lot to visualize the result).

The idea is simple: due to the lack of perspective, it looks like object has a same size regardless of depth. If we set focal length f to 0 and set coordinates in camera space directly, we'll obtain just a cube projection which is a square.

```
if (orthogonal) {
            X_cam = X_p_cam;
            Y_cam = Y_p_cam;
        }
```



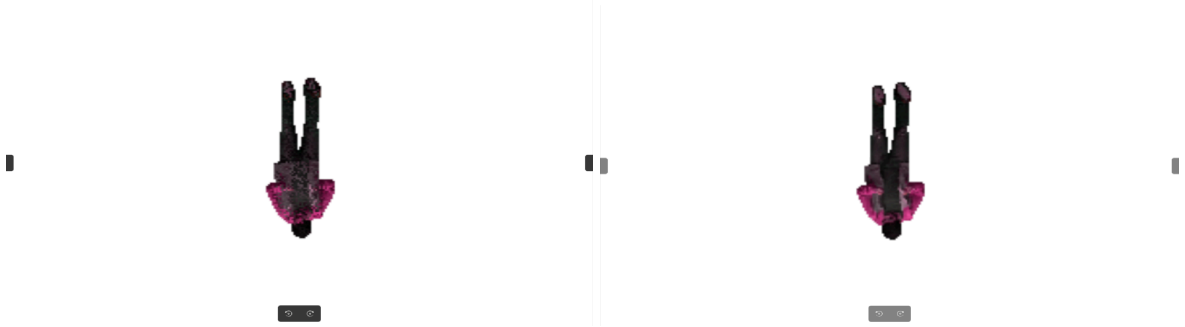
8. Now try the 3D points from file human.off and implement a method that considers occlusion such that only the front can be seen.

We need to take into account a depth buffer to resolve the issue with points overwriting each other. After we allocate memory for the buffer, we paint the image with color and update $Zp'cam$ for every point (to ensure that only the nearest point will be drawn).

```
if (u >= 0 && u < width && v >= 0 && v < height){
            if (use_depth) {
                if (Z_p_cam < z_buffer[u + v*width]) {
                    z_buffer[u + v*width] = Z_p_cam;
                    image.pixmap[u + v*width].red   = points[i].r;
                    image.pixmap[u + v*width].green = points[i].g;
                    image.pixmap[u + v*width].blue  = points[i].b;
                }
            }
```

```
}
```

On the example of file human.off if we set use_depth to 1, we'll see only the visible surface without so much overlapping. On the left, we didn't use Z-buffer and points in the back are coming through the front but on the right, we used the depth and the difference is clear.



9. Provide a set of intrinsic and extrinsic camera parameters in which the effect of use_depth is obvious

For instance, let's call function with the basic configuration but change $\beta = 1.05\ (60\ degrees)$, with use_depth = 0 points from the back overlap with the front points (top image), with use_depth = 1 there is no overlap. With beta parameter we turn cube around Y axis, so some back vertices come through the front pixels.

